Introdução à Biblioteca NumPy

Visão Computacional em Robótica (BLU3040)

Prof. Marcos Matsuo (marcos.matsuo@ufsc.br)

Universidade Federal de Santa Catarina - UFSC

1. O que é NumPy
2. Arrays NumPy
3. Operações Básicas com Arrays
4. Indexação e Fatiamento
5. Manipulação de Formas (Shapes)
6. Álgebra Linear com NumPy
7. Funções Estatísticas
8. Geração de Arrays com Números Aleatórios

O que é NumPy

NumPy é uma biblioteca fundamental para a computação científica em Python.

Ela fornece suporte para arrays e matrizes multidimensionais, além de uma coleção de funções matemáticas para operar nesses arrays.

arquivo exemplo_1.py

```
1  import numpy as np
2
3  # Criação de um array simples
4  arr = np.array([1, 2, 3, 4, 5])
5  print(arr)
```

Arrays NumPy

O que é um array: Um array é uma estrutura de dados que pode armazenar múltiplos valores do mesmo tipo. Em NumPy, arrays são utilizados para realizar operações matemáticas eficientes e são a base para a maioria das operações numéricas.

Criação de arrays: Arrays NumPy podem ser criados a partir de listas ou usando funções integradas do NumPy.

arquivo exemplo_2.py

```
1  import numpy as np
2
3  # Criação a partir de uma lista
4  lista = [1, 2, 3, 4, 5]
5  arr = np.array(lista)
6  print(arr)
```

Arrays NumPy

O que é um array: Um array é uma estrutura de dados que pode armazenar múltiplos valores do mesmo tipo. Em NumPy, arrays são utilizados para realizar operações matemáticas eficientes e são a base para a maioria das operações numéricas.

Criação de arrays: Arrays NumPy podem ser criados a partir de listas ou usando funções integradas do NumPy.

arquivo exemplo_2.py

```
8  # Criação usando funções do NumPy
9  arr_zeros = np.zeros((3, 3))  # Array 3x3 de zeros
10 arr_uns = np.ones((2, 2))     # Array 2x2 de uns
11 arr_range = np.arange(10)     # Array de 0 a 9
12 print(arr_zeros)
13 print(arr_uns)
14 print(arr_range)
```

Arrays NumPy

Propriedades dos Arrays: Os arrays NumPy possuem várias propriedades úteis, como `shape`, `size`, `dtype`, entre outras.

arquivo exemplo_3.py

```
1  import numpy as np
2
3  arr = np.array([[1, 2, 3], [4, 5, 6]])
4
5  print("Forma:", arr.shape)       # (2, 3)
6  print("Tamanho:", arr.size)      # 6
7  print("Tipo:", arr.dtype)        # int64 (ou outro, dependendo do
   ↪  sistema)
```

Operações Básicas com Arrays

Operações Matemáticas: As operações matemáticas em arrays NumPy são realizadas **elemento-a-elemento**, o que significa que a operação é aplicada individualmente a cada par de elementos correspondentes nos arrays.

arquivo exemplo_4.py

```
1  import numpy as np
2
3  a = np.array([1, 2, 3])
4  b = np.array([4, 5, 6])
5
6  # Soma elemento-a-elemento
7  soma = a + b
8  print("Soma elemento-a-elemento:", soma)
9
10 # Subtração elemento-a-elemento
11 subtracao = a - b
12 print("Subtração elemento-a-elemento:", subtracao)
```

Operações Básicas com Arrays

Operações Matemáticas: As operações matemáticas em arrays NumPy são realizadas **elemento-a-elemento**, o que significa que a operação é aplicada individualmente a cada par de elementos correspondentes nos arrays.

arquivo exemplo_4.py

```
14  # Multiplicação elemento-a-elemento
15  multiplicacao = a * b
16  print("Multiplicação elemento-a-elemento:", multiplicacao)
17
18  # Divisão elemento-a-elemento
19  divisao = a / b
20  print("Divisão elemento-a-elemento:", divisao)
```

Operações Básicas com Arrays

Funções Universais (ufuncs): O NumPy fornece funções universais que também realizam operações elemento-a-elemento em arrays. Essas funções são otimizadas para alto desempenho.

arquivo exemplo_5.py

```
1  import numpy as np
2
3  arr = np.array([1, 2, 3, 4, 5])
4
5  print("Seno elemento-a-elemento:", np.sin(arr))
6  print("Logaritmo elemento-a-elemento:", np.log(arr))
7  print("Raiz quadrada elemento-a-elemento:", np.sqrt(arr))
```

Indexação e Fatiamento

Você pode acessar e modificar partes de arrays usando indexação e fatiamento.

arquivo exemplo_6.py

```
1  arr = np.array([10, 20, 30, 40, 50])
2
3  # Indexação
4  print("Primeiro elemento:", arr[0])
5  print("Último elemento:", arr[-1])
6
7  # Fatiamento
8  print("Dois primeiros elementos:", arr[:2])
9  print("Elementos do meio:", arr[1:4])
10 print("Todos os elementos exceto os dois últimos:", arr[:-2])
```

Manipulação de Formas (Shapes)

Você pode alterar a forma de um array sem alterar seus dados com as funções `reshape` e `ravel`.

arquivo exemplo_7.py

```
1  import numpy as np
2
3  arr = np.arange(12)
4  print("Array original:", arr)
5
6  arr_reshape = arr.reshape((3, 4))
7  print("Array reshaped para 3x4:\n", arr_reshape)
8
9  arr_ravel = arr_reshape.ravel()
10 print("Array flatten:\n", arr_ravel)
```

Álgebra Linear com NumPy

NumPy fornece suporte para operações de álgebra linear, como multiplicação de matrizes e resolução de sistemas lineares.

arquivo exemplo_8.py

```
1  import numpy as np
2
3  A = np.array([[1, 2], [3, 4]])
4  B = np.array([[5, 6], [7, 8]])
5
6  # Produto de matrizes
7  produto = np.dot(A, B)
8  print("Produto de Matrizes:\n", produto)
9
10 # Resolução de Sistemas Lineares
11 b = np.array([9, 8])
12 x = np.linalg.solve(A, b)
13 print("Solução do sistema:", x)
```

Funções Estatísticas

NumPy possui várias funções para cálculos estatísticos.

arquivo exemplo_9.py

```
1  import numpy as np
2
3  dados = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
4
5  print("Média:", np.mean(dados))
6  print("Mediana:", np.median(dados))
7  print("Desvio Padrão:", np.std(dados))
```

Geração de Arrays com Números Aleatórios

NumPy fornece funções para gerar arrays com números aleatórios. Essas funções são úteis para simulações e testes estatísticos.

arquivo exemplo_10.py

```
1  import numpy as np
2
3  # Geração de números aleatórios uniformes entre 0 e 1
4  arr_uniform = np.random.rand(3, 3)
5  print("Array com distribuição uniforme [0, 1):\n", arr_uniform)
6
7  # Geração de números aleatórios com distribuição normal (média 0,
   ↪  desvio padrão 1)
8  arr_normal = np.random.randn(3, 3)
9  print("Array com distribuição normal (média 0, desvio padrão 1):\n",
   ↪  arr_normal)
10
11 # Geração de números inteiros aleatórios entre 1 e 10
12 arr_inteiros = np.random.randint(1, 10, (3, 3))
13 print("Array com inteiros aleatórios entre 1 e 10:\n", arr_inteiros)
```